

הסבר מלא על הקוד

Redact-FileContent.ps1

מסמך זה עובר על כל שורה ושורה של הסקריפט, לפי סדר הופעתן בקובץ, ומסביר בעברית מה כל בלוק קוד עושה בפועל וכיצד הוא משתלב בזרימה הכללית של ההרצה - מקבלת הפרמטרים, דרך הדפסת השורות שהשתנו וספירת ההתאמות, ועד דיווח הנתיבים שאין אליהם הרשאת גישה.

505 שורות קוד

21 סעיפים

branch: claude/master

מבנה הסקריפט לפי הסדר

lines 1-72	1	כותרת התייעוד הפנימי (Comment-Based Help)
lines 74-125	2	הגדרת הפרמטרים של הסקריפט (param block)
lines 127-130	3	בדיקת תקינות הנתיב
lines 132-134	4	הכנת תיקיית הגיבוי (רק אם -Backup פעיל)
lines 136-147	5	איסוף כל הקבצים ואיסוף שגיאות הגישה
lines 149-172	6	מבנה הנתונים למיפוי עקבי + הפונקציה Get-PlaceholderName
lines 174-237	7	יצירת חוברת ה-Excel - בניית ה-XML של הגיליונות
lines 239-312	8	יצירת חוברת ה-Excel - חלקי החבילה וכתיבת ה-zip
lines 314-327	9	המונים הגלובליים והפונקציה Write-ChangedLines
lines 329-344	10	תחילת הלולאה הראשית - מעבר על כל קובץ
lines 346-352	11	מעבר שורה-שורה: החלפות מילוליות (\$Replacements)
lines 354-357	12	מחיקת תיאור פורט (-StripPortDescriptions)
lines 359-381	13	שלושת שלבי האנונימיזציה בתוך הלולאה
lines 383-396	14	שמירת השורה שהשתנתה וספירת ההתאמות בה
lines 398-414	15	סיום הלולאה הפנימית וההחלטה אם לכתוב לקובץ
lines 416-424	16	ביצוע הגיבוי בפועל
lines 426-429	17	כתיבת הקובץ המעודכן והדפסת השורות שהשתנו
lines 431-441	18	הדפסת סיכום הרצה כללי + סך ההתאמות
lines 443-457	19	הדפסת טבלאות המיפוי (מה הוחלף במה)
lines 459-488	20	ייצוא טבלאות המיפוי לקובץ Excel
lines 490-505	21	דוח הנתונים שאין אליהם הרשאת גישה

1 כותרת התייעוד הפנימי (Comment-Based Help)

```
1 <#
2 Redact-FileContent.ps1
3
4 Unified replacement/redaction engine for files, recursively through subfolders.
5 All modes below share the same file-processing, dry-run and backup logic.
6
7 1. Literal/regex replacements ($Replacements below) - always applied.
8 Each key is a regex pattern, each value is what to replace it with.
9 (Companion to Search-FilesForWord.ps1 - use that one first to preview matches.)
10
11 2. -AnonymizeSwitchNames - finds "S"/"s" followed by letters/digits/special
```

```

12     characters and ending in "nn" (e.g. S-DC1-01nn), anywhere in a line - including
13     glued to a prefix like "Uplink_to_S-DC1-01nn" - and replaces just that part
14     with a sequential generic placeholder (SWITCH001, SWITCH002, ...).
15
16     3. -AnonymizeUsernames - finds "user" followed directly by digits (e.g.
17         "user22351"), anywhere in a line - including glued to other text like
18         "PC_user22351" or "user10023_workstation" - and replaces just that part with
19         a sequential placeholder (USER001, USER002, ...).
20
21     4. -AnonymizeIPs - finds IPv4 addresses and replaces each with a sequential
22         placeholder (IP001, IP002, ...).
23
24     5. -StripPortDescriptions - removes the description text from interface lines
25         such as "GigabitEthernet1/0/1 - TO_SWITCH001", leaving just the port name
26         ("GigabitEthernet1/0/1"). Covers GigabitEthernet, TenGigabitEthernet,
27         TwentyFiveGigabitEthernet, FastEthernet and Ethernet ports.
28
29     For modes 2-4, the same original value always maps to the same placeholder
30     within a single run (each category has its own counter), but nothing is
31     persisted between runs - running the script again assigns fresh numbers.
32
33     All write modes support -DryRun (report what would change, touch nothing) and
34     -Backup (copy each modified file's original content aside before overwriting it).
35
36     REPORTING:
37
38     * Every changed line is printed under the file it belongs to, with its line
39       number, how many matches it contained, and the line before ("-") and after
40       ("+") the replacement.
41
42     * Each file reports its match count, and the run ends with the total number of
43       matches and changed lines across all files.
44
45     * Folders and files that could not be opened (typically "access denied") are
46       not skipped silently - they are listed at the end with their path and reason.
47
48     * -ExportMappings writes the anonymization mappings to an Excel workbook with
49       three tabs - Switches, Users and IP Addresses - each listing the original
50       value against the generic name it was replaced with. The workbook is built
51       as Open XML directly, so neither Excel nor any module has to be installed.
52
53     USAGE EXAMPLES:
54
55     # Preview the literal replacements only, no files modified
56     .\Redact-FileContent.ps1 -Path "D:\\" -DryRun
57
58     # Apply literal replacements for real, keeping backups
59     .\Redact-FileContent.ps1 -Path "D:\\" -Backup
60
61     # Anonymize switch names, usernames and IPs, and strip port descriptions
62     .\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeSwitchNames -AnonymizeUsernames -
63     AnonymizeIPs -StripPortDescriptions -Backup
64
65     # Anonymization only - skip the literal $Replacements table
66     .\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeIPs -Replacements @{} -DryRun
67
68     # Limit to specific file types
69     .\Redact-FileContent.ps1 -Path "D:\Logs" -Include *.log,*.txt -AnonymizeUsernames
70     s -DryRun
71
72     # Anonymize everything and save the mapping tables as an Excel workbook
73     .\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeSwitchNames -AnonymizeUsernames -
74     AnonymizeIPs -ExportMappings
75
76     # Same, choosing where the workbook is written
77     .\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeIPs -ExcelPath "C:\Temp\mappings.
78     xlsx" -DryRun
79     #>

```

הבלוק בין <# ל-#> הוא הערת תיעוד (block comment) שלא מתבצעת בזמן ריצה. היא מסבירה למי שפותח את הקובץ, לפני שהוא בכלל מריץ אותו, מה הסקריפט עושה:

- הסקריפט הוא "מנוע החלפה/מחיקת מידע" (Redaction) אחד שרץ רקורסיבית על כל תיקיית משנה תחת נתון. נתון.
- קיימים 5 סוגי פעולות שאפשר להפעיל, כל אחת בנפרד או כמה יחד: החלפות מילוליות (תמיד פעיל), אנונימיזציה של שמות מתגים, אנונימיזציה של שמות משתמשים, אנונימיזציה של כתובות IP, ומחיקת תיאורי פורטים.
- הפסקה REPORTING (שורות 36-44) היא החדשה: היא מתעדת שכל שורה שהשתנתה מודפסת תחת הקובץ שלה, שנספרות כמות ההתאמות לכל קובץ ובסך הכל, ושתיקיות שאין אליהן הרשאת גישה מדווחות בסוף ולא נבלעות בשקט.
- הפסקה REPORTING מתעדת גם את ExportMappings - החדש, שכותב את טבלאות המיפוי לקובץ Excel עם שלושה טאבים - מתגים, משתמשים וכתובות IP.
- בסוף מופיעות דוגמאות שימוש בפועל - איך מריצים את הסקריפט משורת הפקודה עם צירופי פרמטרים שונים.

2 הגדרת הפרמטרים של הסקריפט (param block)

```
74 [CmdletBinding(SupportsShouldProcess = $true)]
75 param(
76     # Root folder to process. Defaults to the current directory.
77     [string]$Path = ".",
78
79     # Optional file name filters (wildcards), e.g. *.txt, *.log, *.csv.
80     # Leave empty to process every file.
81     [string[]]$Include = @("*"),
82
83     # Ordered map of regex pattern -> replacement text, applied on every line.
84     [System.Collections.Specialized.OrderedDictionary]$Replacements = [ordered]@{
85         "ass" = "aaa"
86     },
87
88     # Anonymize switch/device name tokens matching -SwitchNamePattern.
89     [switch]$AnonymizeSwitchNames,
90     [string]$SwitchNamePattern = '[Ss][^\s]*nn(?:\S)',
91     [string]$SwitchNamePrefix = "SWITCH",
92
93     # Anonymize username tokens matching -UsernamePattern.
94     [switch]$AnonymizeUsernames,
95     [string]$UsernamePattern = 'user\d+',
96     [string]$UsernamePrefix = "USER",
97
98     # Anonymize IPv4 addresses.
99     [switch]$AnonymizeIPs,
100    [string]$IPPattern = '(?!<!\d)(?:(<?:25[0-5]|2[0-4]\d|1?\d?\d)\.){3}(<?:25[0-5]|2[0-4]\d|1?\d?\d)(?!<!\d)',
101    [string]$IPPrefix = "IP",
102
103    # Strip the description text off interface lines, e.g.
104    # "GigabitEthernet1/0/1 - TO_SWITCH001" -> "GigabitEthernet1/0/1".
105    [switch]$StripPortDescriptions,
106    [string]$PortDescriptionPattern = '^\\s*(?<port>(?:TwentyFiveGigabitEthernet|TenGigabitEthernet|GigabitEthernet|FastEthernet|Ethernet)\d+(?:/\\d+)*\\s*-\\s*.*$',
107
108    # Report matches and what would change, without modifying any file.
```

```

109 [switch]$DryRun,
110
111 # Copy each file's original content into -BackupFolder before overwriting it.
112 [switch]$Backup,
113
114 # Folder to hold backups when -Backup is used. Defaults to a timestamped
115 # folder created next to -Path. Relative structure under -Path is preserved.
116 [string]$BackupFolder,
117
118 # Write the anonymization mappings to an Excel workbook with three tabs:
119 # Switches, Users and IP Addresses (old value vs. the new generic name).
120 [switch]$ExportMappings,
121
122 # Where to write that workbook. Supplying it implies -ExportMappings.
123 # Defaults to a timestamped .xlsx created next to -Path.
124 [string]$ExcelPath
125 )

```

זוהי ה"חתימה" של הסקריפט - כל הפרמטרים שאפשר להעביר לו כשמריצים אותו. שורה 74 - `[CmdletBinding(SupportsShouldProcess = $true)]` - הופכת את הסקריפט ל-cmdlet "אמיתי" של PowerShell, ובפרט מפעילה תמיכה ב- `-WhatIf` וב- `-Confirm`.

- `-Path` - תיקיית השורש שעליה רצים. ברירת מחדל: התיקייה הנוכחית.
- `-Include` - מסננת קבצים לפי שם/סיומת (למשל `*.txt`). ברירת מחדל: הכל.
- `-Replacements` - טבלה מסודרת שבה כל מפתח הוא תבנית regex וכל ערך הוא הטקסט שיחליף אותה. פעילה תמיד.
- `-AnonymizeSwitchNames` / `-SwitchNamePattern` / `-SwitchNamePrefix` - הדגל שמפעיל אנונימיזציה שמות מתגים, ה-regex שמזהה אותם, והקידומת לשם הגנרי.
- `-AnonymizeUsernames` / `-UsernamePattern` / `-UsernamePrefix` - אותו רעיון עבור שמות משתמשים (`userNNNN`).
- `-AnonymizeIPs` / `-IPPattern` / `-IPPrefix` - אותו רעיון עבור כתובות IPv4.
- `-StripPortDescriptions` / `-PortDescriptionPattern` - דגל שמפעיל מחיקת תיאורי פורטים, וה-regex שמזהה שורת פורט.
- `-DryRun` - מצב "הרצה יבשה": רק מדווח מה היה משתנה, לא נוגע באף קובץ.
- `-Backup` / `-BackupFolder` - האם לגבות כל קובץ שמשתנה לפני הכתיבה, ולאן.
- `-ExportMappings` / `-ExcelPath` - החדשים: האם לכתוב את טבלאות המיפוי לקובץ Excel, ולאיזה נתיב. מתן `-ExcelPath` מפעיל את הייצוא מעצמו.

3 בדיקת תקינות הנתיב

```

127 if (-not (Test-Path -LiteralPath $Path)) {
128     Write-Error "Path not found: $Path"
129     return
130 }

```

לפני שעושים כלום, בודקים עם `Test-Path` שהנתיב שהוזן ב- `-Path` באמת קיים על הדיסק. אם לא - מודפסת שגיאה עם `Write-Error` והפקודה `return` עוצרת את כל הסקריפט מיידית, כדי שלא ננסה להמשיך לעבוד על

4 הכנת תיקיית הגיבוי (רק אם Backup פעיל)

```
132 if ($Backup -and -not $BackupFolder) {
133     $BackupFolder = Join-Path (Resolve-Path -LiteralPath $Path) "_backup_$(Get-Date -For
    mat 'yyyyMMdd_HH:mm:ss')"
134 }
```

אם המשתמש ביקש Backup- אבל לא ציין באופן מפורש Backup-Folder , הסקריפט בונה שם תיקייה אוטומטי: תיקייה חדשה בשם `_backup_` ואחריה תאריך ושעה מדויקים (עד השנייה), שנוצרת ליד הנתיב המקורי. כך כל הרצה עם גיבוי מקבלת תיקיית גיבוי משלה, ולא דורסת גיבוי קודם.

5 איסוף כל הקבצים ואיסוף שגיאות הגישה

```
136 $resolvedRoot = (Resolve-Path -LiteralPath $Path).ProviderPath
137
138 # Enumeration errors are collected instead of being swallowed, so folders that
139 # could not be opened (typically "access denied") are reported at the end
140 # rather than being skipped silently.
141 $allFiles = Get-ChildItem -Path $Path -Include $Include -Recurse -File -ErrorAction SilentlyContinue -ErrorVariable accessErrors
142
143 $ignoreCase = [System.Text.RegularExpressions.RegexOptions]::IgnoreCase
144 $switchNameRegex = [regex]::new($SwitchNamePattern, $ignoreCase)
145 $usernameRegex = [regex]::new($UsernamePattern, $ignoreCase)
146 $ipRegex = [regex]::new($IPPattern)
147 $portDescriptionRegex = [regex]::new($PortDescriptionPattern, $ignoreCase)
```

- שורה 136: `$resolvedRoot` הופך את `-Path` לנתיב מלא ומוחלט. זה נדרש כדי לחשב בהמשך את הנתיב היחסי של כל קובץ בתוך תיקיית הגיבוי.
- שורה 141 - זו השורה ששונתה: `Get-ChildItem` אוסף את כל הקבצים תחת `-Path` רקורסיבית בעזרת `-Recurse` , עם הסינון של `-Include` . הדגל `-File` מוציא תיקיות מהתוצאה.
- בעבר `ErrorAction SilentlyContinue` לבדו גרם לסקריפט "לדלג בשקט" על תיקיות שאין הרשאה לגשת אליהן - כלומר הן פשוט נעלמו מהסריקה בלי שום סימן. עכשיו נוסף `-ErrorVariable accessErrors` : הריצה עדיין לא נעצרת, אבל כל שגיאה נאספת למשתנה `$accessErrors` במקום להימחק, ובסוף ההרצה (סעיף 21) מודפסת ממנה רשימת הנתיבים שנחסמו.
- שורות 143-147: בונים מראש ארבעה אובייקטי `[regex]` מקומפלים - אחד לכל תבנית: מתגים, משתמשים, IP ותיאורי פורטים - כדי לא לקמפל מחדש את אותו regex בכל שורה בכל קובץ. כולם חוץ מזה של IP מקבלים את הדגל `IgnoreCase` .

```

149 # Original value (as first encountered) -> generic placeholder, one map per category.
150 # Consistent for this run only - not persisted between runs.
151 $switchNameMap = [ordered]@{}
152 $usernameMap = [ordered]@{}
153 $ipMap = [ordered]@{}
154 $switchNameIndex = 1
155 $usernameIndex = 1
156 $ipIndex = 1
157
158 function Get-PlaceholderName {
159     param(
160         [string]$OriginalValue,
161         [System.Collections.Specialized.OrderedDictionary]$Map,
162         [string]$Prefix,
163         [string]$IndexVarName
164     )
165
166     if (-not $Map.Contains($OriginalValue)) {
167         $index = Get-Variable -Name $IndexVarName -Scope Script -ValueOnly
168         $Map[$OriginalValue] = "{0}{1:D3}" -f $Prefix, $index
169         Set-Variable -Name $IndexVarName -Scope Script -Value ($index + 1)
170     }
171     return $Map[$OriginalValue]
172 }

```

זהו ה"לב" של מנגנון האנונימיזציה העקבית. לכל אחת משלוש הקטגוריות (מתגים/משתמשים/IP) יש:

- מפה (Ordered Dictionary) - (\$switchNameMap, \$usernameMap, \$ipMap) - ששומרת לכל ערך מקורי את השם הגנרי שהוקצה לו, לאורך כל ההרצה (על פני כל הקבצים, לא רק קובץ אחד).
- מונה (Index) - (\$switchNameIndex) וכו' - כל אחד מתחיל מ-1, ועולה בכל פעם שנתקלים בערך חדש באותה קטגוריה.

הפונקציה Get-PlaceholderName (שורות 158-172) מקבלת ערך מקורי, את המפה המתאימה, קידומת (כמו "SWITCH" (ואת שם המשתנה של המונה. אם הערך עדיין לא קיים במפה - נבנה ממנו שם חדש בפורמט SWITCH001 והמונה עולה ב-1; אם הוא כבר קיים - מוחזר אותו שם גנרי שהוקצה לו קודם, כך שאותו S-DC1-01nn תמיד יקבל את אותו SWITCH001 בכל מקום שהוא מופיע.

```

174 function New-MappingWorkbook {
175     <#
176     Writes a real .xlsx (Open XML) with one worksheet per mapping table.
177     Built directly as a zip package, so it needs neither Excel nor the
178     ImportExcel module - only .NET, which PowerShell always has.
179
180     $Sheets is an array of hashtables: @{ Name; Headers; Rows } where Rows
181     is an array of two-element arrays.
182     #>
183     param(
184         [string]$Path,
185         [object[]]$Sheets
186     )
187

```

```

188 Add-Type -AssemblyName System.IO.Compression -ErrorAction SilentlyContinue
189 Add-Type -AssemblyName System.IO.Compression.FileSystem -ErrorAction SilentlyContinue
190
191 function Format-XmlText {
192     param([string]$Text)
193     return [System.Security.SecurityElement]::Escape([string]$Text)
194 }
195
196 function New-SheetXml {
197     param([object]$Sheet)
198
199     $builder = New-Object System.Text.StringBuilder
200     [void]$builder.Append('<?xml version="1.0" encoding="UTF-8" standalone="yes"?>')
201     [void]$builder.Append('<worksheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main">')
202     # Freeze the header row so long mappings stay readable while scrolling.
203     [void]$builder.Append('<sheetViews><sheetView workbookViewId="0">')
204     [void]$builder.Append('<pane ySplit="1" topLeftCell="A2" activePane="bottomLeft"')
205     state="frozen"/>')
206     [void]$builder.Append('</sheetView></sheetViews>')
207     [void]$builder.Append('<cols><col min="1" max="1" width="42" customWidth="1"/><col min="2" max="2" width="26" customWidth="1"/></cols>')
208     [void]$builder.Append('<sheetData>')
209
210     $rowIndex = 1
211     $allRows = @(, $Sheet.Headers) + @($Sheet.Rows)
212     foreach ($row in $allRows) {
213         $style = if ($RowIndex -eq 1) { ' s="1"' } else { '' }
214         [void]$builder.Append("<row r=\"$RowIndex\">")
215         $columnIndex = 0
216         foreach ($value in $row) {
217             $cellRef = "$([char](65 + $columnIndex))$RowIndex"
218             [void]$builder.Append("<c r=\"$cellRef\" t=\"inlineStr\"$style><is><t xm")
219             l:space="preserve">$(Format-XmlText $value)</t></is></c>")
220             $columnIndex++
221         }
222         [void]$builder.Append('</row>')
223         $RowIndex++
224     }
225     [void]$builder.Append('</sheetData></worksheet>')
226     return $builder.ToString()
227 }
228
229 $sheetTags = ""
230 $sheetRels = ""
231 $contentOverrides = ""
232 for ($i = 0; $i -lt $Sheets.Count; $i++) {
233     $id = $i + 1
234     $sheetTags += "<sheet name=\"$$(Format-XmlText $Sheets[$i].Name)\" sheetId=\"$id"
235     " r:id=\"$rId$id\"/>"
236     $sheetRels += "<Relationship Id=\"$rId$id\" Type=\"$http://schemas.openxmlformats.org/officeDocument/2006/relationships/worksheet\" Target=\"$worksheets/sheet$id.xml\"/>"
237     $contentOverrides += "<Override PartName=\"$xl/worksheets/sheet$id.xml\" Content Type=\"$application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml\"/>"
238 }
239 $stylesId = $Sheets.Count + 1

```

זו הפונקציה החדשה שכותבת את קובץ ה-Excel. היא בונה קובץ `.xlsx` אמיתי בפורמט Open XML ישירות כארכיון zip, ולכן היא לא דורשת שיהיה Excel מותקן על המכונה ולא את המודול `ImportExcel` - רק .NET, שקיים תמיד ב-PowerShell.

- הפונקציה מקבלת שני פרמטרים: `$Path` - לאן לכתוב את הקובץ, ו- `$Sheets` - מערך של טבלאות, כל אחת

עם (Name) שם הטאב, (Headers) שורת הכותרת (ו- Rows) השורות עצמן).

- שורות 188-189 טוענות את הרכיבים של NET. שיודעים לכתוב zip, ו- (Format-XmlText) שורות 191-194) בורחת מתווים כמו & ו- < כדי שערך שמכיל אותם לא ישבור את ה-XML.
- (New-SheetXml) שורות 196-226) בונה את ה-XML של גיליון בודד: הגדרת pane שמקיפה את שורת הכותרת בזמן גלילה, רוחב עמודות קבוע, ואז שורה אחר שורה. כל תא נכתב כ- inlineStr, כלומר הטקסט יושב בתוך התא עצמו ולא בטבלת מחרוזות נפרדת - מה שמפשט מאוד את הקובץ. שורת הכותרת מקבלת "s=1", שהוא הסגנון המודגש.
- שורות 228-237 בונות בלולאה אחת את שלושת הרכיבים שמחברים את הגיליונות לחוברת: התגיות <sheet> בתוך workbook.xml, קשרי ה-Relationship בין החוברת לכל קובץ גיליון, וההצהרות של סוגי התוכן.

8 יצירת חוברת ה-Excel - חלקי החבילה וכתובת ה-zip

```
239     $parts = [ordered]@{
240         "[Content_Types].xml" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
241             '<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types"
242             >' +
243                 '<Default Extension="rels" ContentType="application/vnd.openxmlformats-packa
244                 ge.relationships+xml"/>' +
245                 '<Default Extension="xml" ContentType="application/xml"/>' +
246                 '<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlf
247                 orms-officedocument.spreadsheetml.sheet.main+xml"/>' +
248                 $contentOverrides +
249                 '<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlfor
250                 mats-officedocument.spreadsheetml.styles+xml"/>' +
251                 '</Types>'
252     }
253     "_rels/.rels" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
254         '<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relati
255         onships">' +
256             '<Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocum
257             ent/2006/relationships/officeDocument" Target="xl/workbook.xml"/>' +
258             '</Relationships>'
259     "xl/workbook.xml" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
260         '<workbook xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main"
261         ' +
262         'xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationship
263         s">' +
264             "<sheets>$sheetTags</sheets></workbook>"
265     "xl/_rels/workbook.xml.rels" = '<?xml version="1.0" encoding="UTF-8" standalone=
266     "yes"?>' +
267         '<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relati
268         onships">' +
269             $sheetRels +
270             "<Relationship Id=""rId$stylesId"" Type=""http://schemas.openxmlformats.org/
271             officeDocument/2006/relationships/styles"" Target=""styles.xml""/>" +
272             '</Relationships>'
273     # Two cell formats: 0 = plain, 1 = bold (used for the header row).
274     "xl/styles.xml" = '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>' +
275         '<styleSheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/mai
276         n">' +
277             '<fonts count="2"><font><sz val="11"/><name val="Calibri"/></font>' +
278             '<font><b/><sz val="11"/><name val="Calibri"/></font></font>' +
```

```

270         '<fills count="2"><fill><patternFill patternType="none"/></fill>' +
271         '<fill><patternFill patternType="gray125"/></fill></fills>' +
272         '<borders count="1"><border><left/><right/><top/><bottom/><diagonal/></border>' +
273         '<cellStyleXfs count="1"><xfs numFmtId="0" fontId="0" fillId="0" borderId="0" /></cellStyleXfs>' +
274         '<cellXfs count="2"><xfs numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0" />' +
275         '<xfs numFmtId="0" fontId="1" fillId="0" borderId="0" xfId="0" applyFont="1"/></cellXfs>' +
276         '<cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles>' +
277         '</styleSheet>'
278     }
279     for ($i = 0; $i -lt $Sheets.Count; $i++) {
280         $parts["xl/worksheets/sheet${$i + 1}.xml"] = New-SheetXml -Sheet $Sheets[$i]
281     }
282
283     # .NET resolves a relative path against the process working directory, which
284     # is not PowerShell's current location - always hand it a rooted path.
285     if (-not [System.IO.Path]::IsPathRooted($Path)) {
286         $Path = Join-Path (Get-Location).ProviderPath $Path
287     }
288
289     if (Test-Path -LiteralPath $Path) {
290         Remove-Item -LiteralPath $Path -Force
291     }
292     $encoding = New-Object System.Text.UTF8Encoding($false)
293     $archive = [System.IO.Compression.ZipFile]::Open($Path, [System.IO.Compression.ZipArchiveMode]::Create)
294     try {
295         foreach ($entryName in $parts.Keys) {
296             $entry = $archive.CreateEntry($entryName)
297             $stream = $entry.Open()
298             try {
299                 $writer = New-Object System.IO.StreamWriter($stream, $encoding)
300                 $writer.Write($parts[$entryName])
301                 $writer.Flush()
302                 $writer.Dispose()
303             }
304             finally {
305                 $stream.Dispose()
306             }
307         }
308     }
309     finally {
310         $archive.Dispose()
311     }
312 }

```

כאן מוגדרים כל החלקים של חבילת ה-xlsx, ואז הם נכתבים לקובץ:

- `[Content_Types].xml` מצהיר על סוג התוכן של כל חלק בחבילה - זה החלק הראשון ש-Excel קורא, ובלעדיו הקובץ ייחשב פגום.
- `_rels/.rels` מפנה מהחבילה אל `xl/workbook.xml`, ו-`xl/_rels/workbook.xml.rels` מפנה מהחוברת אל שלושת הגיליונות ואל `styles.xml`.
- `xl/styles.xml` מגדיר שני סגנונות בלבד: רגיל ומודגש. המודגש הוא זה שמשמש את שורת הכותרת בכל טאב.
- שורות 283-287: NET פותר נתיב יחסי מול תיקיית העבודה של התהליך, שאינה בהכרח המיקום הנוכחי של PowerShell - ולכן אם התקבל נתיב יחסי, הוא מומר כאן לנתיב מלא לפני הכתיבה.

- שורות 292-311: נפתח ארכיון zip חדש, וכל חלק נכתב אליו כערך נפרד בקידוד UTF-8 ללא BOM. בלוקי ה-try/finally מבטיחים שה-stream והארכיון ייסגרו גם אם הכתיבה נכשלת באמצע, כדי לא להשאיר קובץ נעול.

9 המונים הגלובליים והפונקציה Write-ChangedLines

```

314 $changedCount = 0
315 $scannedCount = 0
316 $totalMatchCount = 0
317 $totalChangedLines = 0
318
319 function Write-ChangedLines {
320     param([object[]]$Lines)
321
322     foreach ($changedLine in $Lines) {
323         Write-Host ("    line {0} ({1} match(es)):" -f $changedLine.Line, $changedLine.M
atches) -ForegroundColor DarkGray
324         Write-Host ("        - {0}" -f $changedLine.Before) -ForegroundColor DarkRed
325         Write-Host ("        + {0}" -f $changedLine.After) -ForegroundColor DarkGreen
326     }
327 }

```

כאן מרוכזים ארבעת המונים של ההרצה, ומיד אחריהם הפונקציה החדשה שמדפיסה את השורות שהשתנו:

- `$changedCount` - כמה קבצים שונו בפועל; `$scannedCount` - כמה קבצים נסרקו בסך הכל.
- `$totalMatchCount` ו-`$totalChangedLines` הם החדשים: סך כל ההתאמות שנמצאו בכל הקבצים, וסך כל השורות שהשתנו. הם מודפסים בסוף ההרצה (סעיף 16).
- `Write-ChangedLines` (שורות 319-327) מקבלת רשימה של שורות שהשתנו, ולכל אחת מדפיסה שלוש שורות פלט: מספר השורה וכמות ההתאמות בה, ואחריהן השורה לפני השינוי (מסומנת - , באדום) והשורה אחרי השינוי (מסומנת + , בירוק) - בדיוק כמו `diff`.

10 תחילת הלולאה הראשית - מעבר על כל קובץ

```

329 foreach ($fileItem in $allFiles) {
330     $scannedCount++
331
332     $content = Get-Content -LiteralPath $fileItem.FullName -ErrorAction SilentlyContinue
333     -ErrorVariable +accessErrors
334     if ($null -eq $content) {
335         continue
336     }
337
338     $fileMatchCount = 0
339     $fileChangedLines = [System.Collections.Generic.List[object]]::new()
340     $lineNumber = 0
341
342     $updatedContent = foreach ($row in $content) {
343         $lineNumber++
344         $currentRow = $row
345         $script:rowMatchCount = 0

```

הלולאה `foreach ($fileItem in $allFiles)` עוברת על כל קובץ שנאסף בסעיף 5. עבור כל קובץ:

- המונה `$ScannedCount` עולה ב-1.
- שורה 332: `Get-Content` קוראת את הקובץ כמערך שורות. גם כאן נוסף `-ErrorVariable +accessErrors` - הסימן + אומר "הוסף לרשימה הקיימת", כך שגם קובץ בודד שנעול או חסום להרשאות נאסף לאותו דוח בסוף, ולא רק תיקיות.
- אם הקריאה נכשלה (`$content` ריק/`continue` - null מדלג לקובץ הבא בלי לגעת בו.
- שורות 337-339 מאתחלים לכל קובץ שלושה משתנים: `$fileMatchCount` - מונה ההתאמות בקובץ; `$fileChangedLines` - רשימת השורות שהשתנו בקובץ, חדש; ו- `$lineNumber` - מונה מספר השורה בתוך הקובץ, חדש גם הוא, כדי שאפשר יהיה להדפיס לכל שינוי את מספר השורה שלו.
- שורה 344: `$script:rowMatchCount = 0` מאפס את מונה ההתאמות של השורה הנוכחית. זה מונה חדש ברמת השורה (ולא רק ברמת הקובץ), והוא ב-scope של הסקריפט כדי שגם ה-script-blocks של האנונימיזציה יוכלו להעלות אותו.

11 מעבר שורה-שורה: החלפות מילוליות (\$Replacements)

```

346 foreach ($search in $Replacements.Keys) {
347     $lineMatches = [regex]::Matches($currentRow, $search).Count
348     if ($lineMatches -gt 0) {
349         $script:rowMatchCount += $lineMatches
350         $currentRow = $currentRow -replace $search, $Replacements[$search]
351     }
352 }
```

עבור כל שורה בקובץ עוברים על כל תבנית ב- `$Replacements`. לכל תבנית: סופרים כמה פעמים היא מופיעה בשורה עם `[regex]::Matches(...).Count`, מוסיפים את המספר הזה למונה של השורה (`$script:rowMatchCount`, ורק אם יש לפחות התאמה אחת - מבצעים בפועל את ההחלפה עם האופרטור `-replace`).

12 מחיקת תיאור פורט (\$StripPortDescriptions)

```

354 if ($StripPortDescriptions -and $portDescriptionRegex.IsMatch($currentRow)) {
355     $script:rowMatchCount++
356     $currentRow = $portDescriptionRegex.Replace($currentRow, '{port}')
357 }
```

אם הדגל `$StripPortDescriptions` פעיל, בודקים אם השורה כולה תואמת את `$PortDescriptionPattern` (למשל `GigabitEthernet1/0/1 - TO_SWITCH001`). אם כן - מעלים את מונה השורה ב-1, ומחליפים את כל השורה בקבוצת ה- `(?<port>...)` שנתפסה, כלומר משאירים רק את שם הפורט ומוחקים את המקף ואת הטקסט שאחריו.

השלב הזה מתבצע לפני שלושת שלבי האנונימיזציה, כך שאם גם `$StripPortDescriptions` וגם `$AnonymizeSwitchNames` פעילים - קודם נמחק ה-description, ורק מה שנשאר בשורה עובר הלאה לבדיקת האנונימיזציה.

```

359     if ($AnonymizeSwitchNames) {
360         $currentRow = $switchNameRegex.Replace($currentRow, {
361             param($match)
362             $script:rowMatchCount++
363             Get-PlaceholderName -OriginalValue $match.Value -Map $switchNameMap -Pre
fix $SwitchNamePrefix -IndexVarName "switchNameIndex"
364         })
365     }
366
367     if ($AnonymizeUsernames) {
368         $currentRow = $usernameRegex.Replace($currentRow, {
369             param($match)
370             $script:rowMatchCount++
371             Get-PlaceholderName -OriginalValue $match.Value -Map $usernameMap -Prefi
x $UsernamePrefix -IndexVarName "usernameIndex"
372         })
373     }
374
375     if ($AnonymizeIPs) {
376         $currentRow = $ipRegex.Replace($currentRow, {
377             param($match)
378             $script:rowMatchCount++
379             Get-PlaceholderName -OriginalValue $match.Value -Map $ipMap -Prefix $IPP
refix -IndexVarName "ipIndex"
380         })
381     }

```

שלושה בלוקים כמעט זהים במבנה - אחד לכל קטגוריה (מתגים, משתמשים, IP). כל בלוק פעיל רק אם הדגל המתאים דלוק. בכל בלוק:

- קוראים ל- `$xxxRegex.Replace($currentRow, {...})` - הפעולה `.Replace`. מקבלת `script-block` `MatchEvaluator` (במקום מחרוזת קבועה, כלומר לכל התאמה מריצים פונקציה שמחליטה מה יהיה הטקסט המחליף).
- בתוך אותו `script-block`, `$script:rowMatchCount++` מעלה עכשיו את מונה ההתאמות של השורה (בעבר זה היה מונה הקובץ), ואז `Get-PlaceholderName` מחזירה את השם הגנרי העקבי שיחליף את הטקסט המקורי.
- שלושת השלבים רצים לפי הסדר: קודם שמות מתגים, אחר כך שמות משתמשים, ולבסוף כתובות IP - כל אחד פועל על התוצאה של הקודם לו באותה שורה.

```

383     # Keep the line itself, so every change can be printed with its line
384     # number and how many matches it contained.
385     if ($script:rowMatchCount -gt 0) {
386         $fileMatchCount += $script:rowMatchCount
387         $fileChangedLines.Add([PSCustomObject]@{
388             Line      = $lineNumber
389             Matches    = $script:rowMatchCount
390             Before     = $row.Trim()
391             After      = $currentRow.Trim()
392         })

```

```

393     }
394
395     $currentRow
396 }

```

זהו הבלוק החדש שמאפשר להדפיס בפועל את השורות. אחרי שכל ההחלפות רצו על השורה, בודקים אם `$script:rowMatchCount` גדול מאפס - כלומר האם השורה הזו בכלל השתנתה:

- אם כן, מוסיפים את מונה השורה למונה הקובץ (`$fileMatchCount += $script:rowMatchCount`).
- ואז שומרים ב- `$fileChangedLines` אובייקט עם ארבעה שדות: `Line` - מספר השורה בקובץ; `Matches` - כמה התאמות היו בשורה; `Before` - השורה המקורית; ו- `After` - השורה אחרי ההחלפה. זה מה שמאפשר להדפיס אחר כך את השורה עצמה, ולא רק את שם הקובץ.
- שורה 395: `$currentRow` מוחזרת מהלולאה הפנימית ונאספת ל- `$updatedContent` - כך שבסוף יש לנו את כל הקובץ מחדש, שורה אחר שורה, עם כל השינויים.

15 סיום הלולאה הפנימית וההחלטה אם לכתוב לקובץ

```

398     if ($fileMatchCount -eq 0) {
399         continue
400     }
401
402     $changedCount++
403     $totalMatchCount += $fileMatchCount
404     $totalChangedLines += $fileChangedLines.Count
405
406     if ($DryRun) {
407         Write-Host "[DryRun] Would update: $($fileItem.FullName) ($fileMatchCount match(
408 es) on $($fileChangedLines.Count) line(s))" -ForegroundColor DarkYellow
409         Write-ChangedLines -Lines $fileChangedLines
410         continue
411     }
412     if (-not $PSCmdlet.ShouldProcess($fileItem.FullName, "Replace $fileMatchCount match(
413 es)")) {
414         continue
415     }

```

- שורות 398-400: אם אף שורה בקובץ לא השתנתה (`$fileMatchCount -eq 0`) - מדלגים לקובץ הבא. הקובץ לא נכתב מחדש, לא מגובה ולא נספר כ"שונה".
- שורות 402-404: אחרת, מונה הקבצים ששונו עולה ב-1, ובנוסף מתעדכנים שני המונים הכלליים החדשים - `$totalMatchCount` (סך ההתאמות) ו- `$totalChangedLines` (סך השורות שהשתנו).
- שורות 406-410: במצב `-DryRun` מודפסת שורת דיווח כתומה עם שם הקובץ, מספר ההתאמות ומספר השורות שהשתנו, ומיד אחריה `Write-ChangedLines` מדפיסה את כל השורות עצמן (לפני/אחרי) - כך שאפשר לראות בדיוק מה היה משתנה, בלי לגעת באף קובץ.
- שורות 412-414: `$PSCmdlet.ShouldProcess(...)` הוא המנגנון הפנימי של PowerShell שמפעיל את `-WhatIf / -Confirm`. אם המשתמש הריץ עם `-WhatIf`, השורה הזו תחזיר `$false` והקובץ לא ייכתב - בלי צורך בקוד נוסף.

```

416     if ($Backup) {
417         $relativePath = $fileItem.FullName.Substring($resolvedRoot.Length).TrimStart('\', '/')
418         $backupPath = Join-Path $BackupFolder $relativePath
419         $backupDir = Split-Path -Path $backupPath -Parent
420         if (-not (Test-Path -LiteralPath $backupDir)) {
421             New-Item -ItemType Directory -Path $backupDir -Force | Out-Null
422         }
423         Copy-Item -LiteralPath $fileItem.FullName -Destination $backupPath -Force
424     }

```

אם `-Backup` פעיל: מחשבים את הנתיב היחסי של הקובץ ביחס לתיקיית השורש (`$resolvedRoot` שחושבה בשורה 136), ובונים ממנו נתיב מקביל בתוך תיקיית הגיבוי - כך שמבנה התיקיות המקורי נשמר גם בגיבוי. אם תיקיית היעד עדיין לא קיימת, `New-Item -ItemType Directory` יוצרת אותה כולל כל תיקיות הביניים (החסרות) בזכות `-Force`. לבסוף `Copy-Item` מעתיקה את הקובץ המקורי, לפני השינוי, למיקום הגיבוי.

17 כתיבת הקובץ המעודכן והדפסת השורות שהשתנו

```

426     Write-Host "Working on: $($fileItem.FullName) ($fileMatchCount match(es) on $($fileC
427         hangedLines.Count) line(s))" -ForegroundColor DarkYellow
428     Write-ChangedLines -Lines $fileChangedLines
429     $updatedContent | Set-Content -LiteralPath $fileItem.FullName

```

שורה 426 מדפיסה שורת סטטוס צהובה עם שם הקובץ, מספר ההתאמות שנמצאו בו וכמה שורות השתנו. שורה 269 היא החדשה: `Write-ChangedLines` מדפיסה מיד אחריה את כל השורות של אותו קובץ - לכל אחת מספר שורה, כמות התאמות, והשורה לפני ואחרי. שורה 428, `Set-Content`, כותבת את `$updatedContent` בחזרה לאותו קובץ. זהו הרגע היחיד שבו קובץ באמת משתנה על הדיסק.

18 הדפסת סיכום הרצה כללי + סך ההתאמות

```

431 Write-Host ""
432 if ($DryRun) {
433     Write-Host "Dry run complete. $changedCount of $scannedCount file(s) would be update
434     d." -ForegroundColor Green
435 } else {
436     Write-Host "Complete successfully! $changedCount of $scannedCount file(s) updated."
437     -ForegroundColor Green
438     if ($Backup) {
439         Write-Host "Backups saved under: $BackupFolder" -ForegroundColor Green
440     }
441 }
442 Write-Host "Total matches found: $totalMatchCount on $totalChangedLines line(s)." -Foreg
443     roundColor Green

```

אחרי שהלולאה על כל הקבצים הסתיימה, מודפס סיכום: במצב `-DryRun` - כמה קבצים היו משתנים מתוך כמה שנסרקו; אחרת - כמה קבצים אכן שונו, ואם היה גיבוי גם הנתיב המלא לתיקיית הגיבוי.

שורה 441 היא החדשה: `Total matches found: N on M line(s)` - סך כל ההתאמות שנמצאו בכל הקבצים יחד, וסך כל השורות שהשתנו. זו התשובה לשאלה "כמה פעמים המילה נמצאה" ברמת ההרצה כולה, בנוסף לספירה שמודפסת לכל קובץ בנפרד.

19 הדפסת טבלאות המיפוי (מה הוחלף במה)

```
443 function Write-NameMapping {
444     param([string]$Title, [System.Collections.Specialized.OrderedDictionary]$Map)
445
446     if ($Map.Count -eq 0) {
447         return
448     }
449     Write-Host "`n$Title mapping for this run:" -ForegroundColor Cyan
450     $Map.GetEnumerator() | ForEach-Object {
451         Write-Host (" {0} -> {1}" -f $_.Key, $_.Value)
452     }
453 }
454
455 if ($AnonymizeSwitchNames) { Write-NameMapping -Title "Switch name" -Map $switchNameMap }
456 if ($AnonymizeUsernames) { Write-NameMapping -Title "Username" -Map $usernameMap }
457 if ($AnonymizeIPs) { Write-NameMapping -Title "IP address" -Map $ipMap }
```

הפונקציה `Write-NameMapping` (שורות 443-453) מקבלת כותרת ומפה, ואם יש בה לפחות ערך אחד - מדפיסה כותרת כחולה ואז שורה לכל ערך מקורי עם השם הגנרי שהוקצה לו (למשל `S-DC1-01nn -> SWITCH001`). שלוש השורות 455-457 קוראות לפונקציה פעם אחת לכל קטגוריה שהייתה פעילה, כדי שבסוף ההרצה יהיה למפעיל תיעוד מלא של מה הוחלף במה - גם אם המיפוי עצמו לא נשמר לקובץ ולא יישמש בהרצה הבאה.

20 ייצוא טבלאות המיפוי לקובץ Excel

```
459 # Excel workbook with one tab per mapping table, so the "who became what" list
460 # survives the run instead of only scrolling past in the console.
461 if ($ExportMappings -or $ExcelPath) {
462     if (-not $ExcelPath) {
463         $ExcelPath = Join-Path (Resolve-Path -LiteralPath $Path) "Redaction-Mappings_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').xlsx"
464     }
465
466     $mappingSheets = @(
467         @(
468             @{ Name = "Switches"; Headers = @("Original Switch Name", "New Name"); M
469 ap = $switchNameMap }
469             @{ Name = "Users"; Headers = @("Original User Name", "New Name"); M
470 ap = $usernameMap }
470             @{ Name = "IP Addresses"; Headers = @("Original IP Address", "New Name"); M
471 ap = $ipMap }
471         ) | ForEach-Object {
472             @{
473                 Name = $_.Name
474                 Headers = $_.Headers
475                 Rows = @($_.Map.GetEnumerator() | ForEach-Object { , @($_.Key, $_.Value) })
476             }
477         }
478     )
479 }
```

```

479
480     try {
481         New-MappingWorkbook -Path $ExcelPath -Sheets $mappingSheets
482         $rowTotal = ($mappingSheets | ForEach-Object { @($_.Rows).Count } | Measure-Obje
ct -Sum).Sum
483         Write-Host "`nMapping workbook saved to: $ExcelPath ($rowTotal mapping(s) across
3 tabs)" -ForegroundColor Cyan
484     }
485     catch {
486         Write-Warning "Could not write the mapping workbook to '$ExcelPath': $($_.Except
ion.Message)"
487     }
488 }

```

זהו החלק שמפעיל בפועל את יצירת קובץ ה-Excel, אחרי שכל ההחלפות כבר רצו והמיפויים מלאים:

- הייצוא מתבצע אם הופעל `-ExportMappings` או אם ניתן `-ExcelPath`. אם לא נבחר נתיב, הקובץ נשמר ליד תיקיית היעד בשם `Redaction-Mappings` עם חותמת תאריך ושעה, כך שכל הרצה מקבלת קובץ משלה.
- שורות 464-476 מרכיבות את שלושת הטאבים לפי הסדר: `Switches` - שם מתג מקורי מול השם החדש; `Users` - שם משתמש מקורי מול השם החדש; ו- `IP Addresses` - כתובת ה-IP המקורית מול השם שניתן לה. כל טאב נבנה מהמפה המתאימה שנצברה במהלך ההרצה.
- שלושת הטאבים נוצרים תמיד, גם אם קטגוריה מסוימת לא הופעלה בהרצה - במקרה כזה הטאב שלה יכיל רק את שורת הכותרת.
- הקריאה עטופה ב- `try/catch`: אם הכתיבה נכשלת (למשל הקובץ פתוח כרגע ב-Excel או אין הרשאת כתיבה לתיקייה), מודפסת אזהרה בלבד - אבל כל עבודת ההחלפה שכבר בוצעה על הקבצים לא הולכת לאיבוד.

21 דוח הנהיגים שאין אליהם הרשאת גישה

```

490 # Folders and files that could not be opened, with the reason. TargetObject holds
491 # the path for these provider errors; fall back to the error's target name.
492 $inaccessible = $accessErrors |
493     ForEach-Object {
494         [PSCustomObject]@{
495             Path = if ($_.TargetObject) { $_.TargetObject } else { $_.CategoryInfo.Tar
getName }
496             Reason = $_.Exception.Message
497         }
498     } |
499     Where-Object { $_.Path } |
500     Sort-Object Path -Unique
501
502 if ($inaccessible) {
503     Write-Host "`nPaths that could not be accessed and were skipped: $($($inaccessible).
Count)" -ForegroundColor Yellow
504     $inaccessible | Format-Table -AutoSize -Wrap
505 }

```

זהו הסעיף החדש שסוגר את הפער: כל השגיאות שנאספו ל- `$accessErrors` גם מ- `Get-ChildItem` בשורה 141 וגם מ- `Get-Content` בשורה 332 (מומרות כאן לרשימה קריאה).

- לכל שגיאה בונים אובייקט עם שני שדות: `Path` - הנהיג שנחסם, ו- `Reason` - הודעת השגיאה עצמה (בדרך כלל `Access to the path ... is denied`).

- בשגיאות מסוג זה הנתיב יושב ב- `$_TargetObject` ; אם משום מה הוא ריק, נופלים חזרה ל- `$_CategoryInfo.TargetName`.
- `Where-Object { $_.Path }` מסנן שגיאות בלי נתיב, ו- `Sort-Object Path -Unique` ממין ומוריד כפילויות, כדי שאותה תיקייה לא תופיע פעמיים.
- אם נשאר משהו ברשימה, מודפסת כותרת צהובה עם מספר הנתיבים שנחסמו ואחריה טבלה של נתיב + סיבה. כך ברור בסוף כל הרצה אם הסריקה הייתה מלאה, או שהיו תיקיות שהסקריפט לא הצליח להיכנס אליהן.